

SetuBiz

Team Study Guide

AI-Driven Hyper-Local Business Advisory and Financial Structuring Assistant for Rural Micro-Entrepreneurs

Ministry of Social Justice and Empowerment | Software | Theme: Agriculture, FoodTech and Rural Development

Contents

1. The 60-second version
2. What the problem statement actually wants
3. Architecture: the three-layer doctrine
4. Datasets: how we hit zero synthetic data
5. Mathematical models: every formula, with worked numbers
6. Features, flows, and the fast-response granularities
7. What we did extra, and why it shines
8. Honest gaps and judge questions
9. Appendix: file map, glossary, demo script

How to read this guide. This is the conceptual and technical map of everything we built, written so that any teammate can explain any part of the system to a judge without opening the code. Anything that comes from the design document (SIH26091_Models_Architecture_Workflow.docx) but is not in the code is tagged:

- **[Built]:** it exists in the codebase today, tested and demoable.
- **[Planned]:** it is described in the design document and intended, but it is not in the code. **Never present a [Planned] item as working.**

Formulas and constants are quoted from the actual code with file references. The running example throughout is our demo persona: **a village near Ranchi, ₹1,00,000 in savings, wanting a 2-animal dairy unit.**

1. The 60-second version

The thesis. The problem statement's own formula, $Project\ Cost = Margin \div 10\%$, computes the **maximum** a person may borrow. Maximum borrowing is precisely the mechanism behind the rural enterprise failures the ministry is trying to prevent. SetuBiz answers a different question: not "*what is the most you can borrow?*" but "*what is the most you can actually repay?*" It shows both numbers side by side.

The demo moment (memorise these numbers):

	Scheme formula (maximum)	SetuBiz recommendation
Loan	₹9,00,000	₹2,05,000
Worst-year DSCR	0.52 (fails)	2.30 (comfortable)
What binds it	nothing, it is the ceiling	the -15% revenue stress test
Why	the 10% margin rule turns ₹1L of savings into a ₹10L project and a ₹9L loan	the cash flow of a 2-animal dairy survives a bad year only up to ₹2.05L

And the honest kicker: the unit actually costs **₹4,10,800**, of which ₹3,10,800 needs borrowing, so the real advice is "a smaller unit, or ₹48,800 more of your own money", which the app computes too.

The architecture doctrine, in three sentences. Machine learning is used only where the data is genuinely uncertain, namely competitors and demand. Every financial calculation and every eligibility verdict is deterministic, auditable code, because a loan number must be verifiable, not predicted. The language model explains decisions made elsewhere; it never makes one.

The data doctrine, in one sentence. Zero synthetic datasets: every number on screen traces to Census 2011, SHRUG, Mission Antyodaya, Economic Census 2013, HCES 2023-24, OpenStreetMap, Agmarknet or official scheme portals, and the UI proves it with per-figure provenance.

2. What the problem statement actually wants

2.1 The literal requirements

Module 1, the Hyper-Local Business Feasibility Report, generated from regional demographic and economic data across exactly **six parameters**:

1. **Market Reach**: the consumer base within a 5 to 10 km radius, plus distribution channels.
2. **Opportunity Analysis**: unserved and underserved niches in the regional market.
3. **SWOT Analysis**: strengths, weaknesses, opportunities and threats tailored to budget constraints.
4. **Threats Identification**: localised risks such as supply chain issues, seasonal swings and single-buyer dependence.
5. **Competitor Mapping**: business density in the local block.
6. **Product Market Value**: optimal pricing based on local purchasing power.

Module 2, the Smart Financial Calculator and Scheme Router, driven by the *Available Margin Capital*, which represents 10% of the total project cost:

Financial parameter	PS rule	Margin of ₹1,00,000 example
Total Project Cost	Margin ÷ 10%	₹10,00,000
Maximum Loan	90% of project cost	₹9,00,000
Logic A (Micro Finance Scheme)	Project Cost up to ₹1.40 L: 6.5% interest, 3-year tenure, 3-month moratorium	
Logic B (Term Loan Scheme)	Project Cost above ₹1.40 L and up to ₹50.00 L: 8.0% interest, 7-year tenure, 6-month moratorium	
Repayment generator	quarterly schedules, operating cost projections, working capital, moratorium-aware	

2.2 Reading between the lines: what MoSJE is really asking

The stated **impact goals** are the tell:

- "Reduce failure rates of rural micro-enterprises through data-backed business viability checks."
- "Eliminate financial confusion regarding margin requirements and loan eligibility."
- "Empower marginalized youth with grass-root enterprise intelligence."

A team that builds a calculator implementing the PS formula verbatim has built the thing that produces the failures. It tells a dairy farmer with ₹1L of savings that she may borrow ₹9L against a business earning ₹93,600 a year before expenses. Her worst-year debt coverage would be 0.52. That loan defaulting is *not* her failure; it is the advisory's failure.

So the PS actually wants four things:

1. **Viability before eligibility**. The feasibility report must feed the loan sizing, not sit beside it. (In ours it does: the feasibility numbers for revenue, opex and catchment are the same inputs the DSCR engine uses.)
2. **Every number with a source**. "Data-backed" means a judge can ask *where did that number come from?* and get a dataset name. (Ours can, figure by figure; see section 7.)

3. **Margin and eligibility confusion eliminated.** The app must speak in the user's terms: what you have, what the unit costs, what you can carry, which corporation's form to fill, and which documents to bring. (Ours routes Scheduled Caste applicants to NSFDC, safai karamcharis to NSKFDC and OBC or EBC applicants to NBCFDC, and prints the document checklist and the Jharkhand SCA's address.)
4. **Built for the marginalized.** The target user often cannot fill a web form. Voice input, Hindi output and a printable sheet for a bank visit are not extras; they are the accessibility layer that makes the advisory real.

2.3 Impact goals and the features that serve them

Impact goal	Feature that serves it	Where
Reduce failure rates	DSCR of at least 1.5 right-sizing, plus -15% and -30% stress tests, plus the maximum-versus-recommended comparison	<code>finance/rightsizing.py</code> , <code>LoanComparison.tsx</code>
Viability checks	The six-parameter feasibility computed from real data, always as ranges	<code>feasibility/</code> , section 6.2
End margin confusion	The PS router <i>plus</i> "the unit actually costs X, you need Y, borrow Z"	<code>finance/router.py</code> , <code>facts/builder.py</code>
Loan eligibility clarity	Deterministic corporation routing, income ceilings, document checklist, SCA address	<code>eligibility/rules.py</code>
Grass-root intelligence	29,480 real Jharkhand villages, block-level densities, state consumption profiles	<code>data/real/</code>
Accessibility	Voice input (Hindi), bilingual output, a print and PDF handoff sheet, 44px touch targets	<code>MicButton.tsx</code> , <code>app.css</code>

3. Architecture: the three-layer doctrine

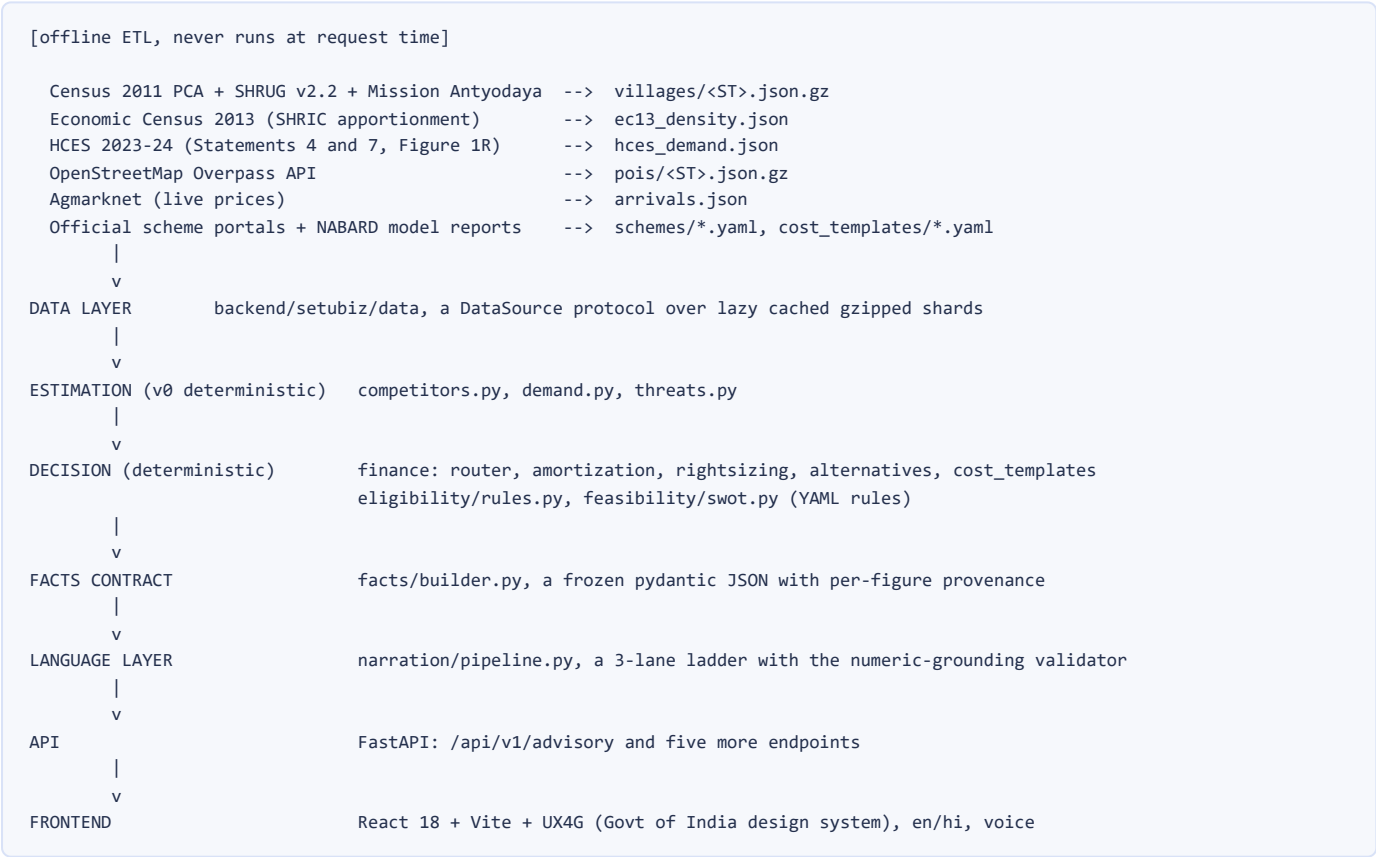
3.1 Why three layers

The predictable failure mode for this PS is an LLM wrapped around a chat box, inventing market figures and eligibility verdicts. A ministry judge will ask "where did that number come from?" and that project ends there. The design document's guiding principle, which we implemented, is:

Layer	Responsibility	Implemented as
Estimation layer	Quantities nobody observes directly: how many competitors exist, how much local demand there is	Deterministic estimators v0 (EC13 density z-scores, HCES-share bands); trained ML models are the designed upgrade path
Decision layer	Loan structuring, right-sizing, eligibility, referrals, stress tests	Pure deterministic code: <code>Decimal</code> arithmetic, rule engines, YAML rules. No ML, ever.
Language layer	Explaining, translating, phrasing	A narrator ladder (template first, then local Llama, then cloud LLM), with every lane validated against the facts object

The language layer **never decides anything**. It receives a locked facts object and is forbidden from introducing any figure that is not in it.

3.2 The repo's actual layer map



[Planned] upgrades that drop into this map: the `LightgbmDensityEstimator` (Model 1, already registered in `competitors.py`, currently raises `NotImplementedError`; swapping estimators is a one-line environment change, `SETUBIZ_COMPETITOR_ESTIMATOR`), the Fay-Herriot demand model, IndicConformer ASR with Silero VAD and Indic-TTS, and PostGIS with pgvector behind a RAG store.

3.3 Request flow: one POST, no streaming

1. The frontend wizard collects the village (confirmed from the top three candidates), the radius (3 to 30 km), the business category (fetched live from the cost-templates endpoint), savings, social category, income and the woman and experience flags, plus the language.
2. A single POST to `/api/v1/advisory` runs `build_facts()`, the whole deterministic pipeline, in about 490 ms on real Jharkhand data: resolve the village, compute market reach, estimate competitors, demand and threats, route the scheme, right-size the loan, generate alternatives, amortize the maximum loan, the recommended loan and the alternate moratorium mode, assess eligibility, compute SWOT metrics, build the numeric index, attach provenance and freeze.
3. `narrate(facts, language)` runs the narrator ladder and passes every lane's output through the numeric-grounding validator.
4. One response returns `{ facts, report, validation }`. The frontend renders sections; it never re-derives numbers and never translates prose, because the server narrates per language.

3.4 Model inventory (know this cold)

#	Model or component	Status	Role
1	Competitor Density Estimator (LightGBM, planned) over the EC13 z-score estimator	[Planned] / [Built]	How many rivals are nearby
2	Local Demand Estimator (Fay-Herriot planned, HCES-share band built)	[Planned] / [Built]	Local market size in ₹
3	Intent and slot classifier over IndicBERT or LaBSE	[Planned]	Speech to structured fields
4	IndicConformer or IndicWhisper ASR (INT8)	[Planned] (the browser Web Speech API is [Built])	Speech to text
5	Indic-TTS or Piper	[Planned]	Reads the report aloud
6	Silero VAD	[Planned]	End-of-utterance detection
7	BGE-M3 with the bge-reranker (RAG)	[Planned]	Scheme document retrieval
8	Llama 3.1 8B Instruct Q4_K_M (llama.cpp with GBNF)	[Built] (lane exists, off by default, weights not shipped)	Explains only
9	Cloud LLM narration (schema-constrained)	[Built]	Explains only
10	Numeric-grounding validator with the catch-rate metric	[Built]	Anti-hallucination
11	Jinja template narrator	[Built] , the default lane that always works	Offline narration
12	SARIMA or Prophet price forecaster	[Planned] (stretch)	Seasonality band
13	Financial engine, eligibility rules, SWOT rules, village matcher, alternatives	[Built] and deliberately not ML	The decision layer

4. Datasets: how we hit zero synthetic data

4.1 What the claim means, precisely

- The app **defaults to real data**: `config.py` sets `data_dir` to the real directory, and the API's `/metrics` endpoint reports `data_source: "live"`. Switching to the sample is one environment variable (`SETUBIZ_DATA_DIR`), used only by tests.
- Synthetic data exists in exactly two roles: **pinned test fixtures** (tests must be deterministic, so `conftest.py` pins the sample directory) and an **offline fallback**. It is never hidden: every file carries `_meta.synthetic`, the facts object carries `contains_synthetic_data`, the UI shows a "Demonstration data" banner, and the provenance drawer tags each source "official" or "sample" individually.
- **Hand-curated artifacts are not synthetic data**. The five business cost templates are transcribed and adapted from NABARD model project reports (the same documents bank appraisers use), scheme rules are transcribed from official portals with source URLs, and SWOT rules are editorial thresholds. No number was invented by a generator.
- **Honest omissions**. Agmarknet publishes only the current day's prices, so the seasonality chart is *omitted on live data rather than invented* (it appears on sample data, clearly flagged). That discipline, dropping rather than fabricating, is the whole credibility story.

4.2 Source by source

Source	What it gives us	Granularity	Consumed by	Status
Census 2011 Primary Census Abstract	Households, population, literacy, SC/ST share, workers, per village	Village (29,480 JH villages)	Market reach, demand scaling, SWOT metrics	[Built]
SHRUG v2.2 (devdatalab.org)	Authoritative village IDs (<code>shrid</code>), names, lat/lon centroids, <code>tdist_50</code> (km to the nearest town of 50,000+)	Village	The join key for <i>everything</i> ; coordinates for radius queries	[Built]
Mission Antyodaya 2019	Amenity flags: bank, pucca road, power, mandi, haat, milk route, veterinary, SHGs	Village level for 2019; never fed into the <code>*_2011</code> fields, to avoid double-counting growth	Market reach (banks, roads), threats (single buyer)	[Built]
Economic Census 2013	Enterprise counts per block, mapped to categories via SHRIC codes	Block (259 JH blocks), enterprises per 1,000 households	Competitor mapping	[Built]
HCES 2023-24 (MoSPI)	Rural MPCE per state, household size, category spend shares, CV, income fractile segments	State (29 states and UTs)	Demand estimation, purchasing power, Product Market Value	[Built]
OpenStreetMap (Overpass API)	Existing shops with coordinates, by category tag	Point POIs (120 in rural JH)	The competitor <i>floor</i> ("a POI count is a floor, never a census")	[Built]
Agmarknet (data.gov.in)	Daily mandi prices and arrivals by commodity	Daily, live	Threats (price band); seasonality needs history we do not have	[Built] (live prices only)

Source	What it gives us	Granularity	Consumed by	Status
NSFDC, NSKFC and NBCFDC portals	Scheme rules verbatim: rates, tenures, moratoria, caps, eligibility, ceilings	Scheme level, transcribed with source URLs	Scheme router, eligibility engine, RAG corpus (planned)	[Built]
NABARD model project reports	Unit economics: line items, revenue and opex norms per business	Five templates, bilingual	Cost templates, then required capital, NOI and DSCR	[Built] (hand-curved)
Udyam MSME registrations	Registered enterprise counts by district or pincode with activity	District or pincode	Training labels for Model 1	[Planned] ; the granularity check is the first roadmap task
Intercensal scaling	District growth factors (Ranchi 1.29, default 1.25) to carry 2011 figures forward to 2026	District	Household scaling	[Built]

4.3 Why each dataset is needed (the dependency chain)

Follow one number, the **recommended loan**, backwards:

```

₹2,05,000 recommended
  <- DSCR at least 1.5, and stressed DSCR at least 1.0, against an annual NOI of ₹93,600
    <- revenue of 600 litres at ₹46 per litre (NABARD template) minus opex of ₹21,600 a month
      <- milk price benchmarked from mandi prices (Agmarknet)
      <- yield of 10 litres per animal per day over a 300-day lactation (NABARD norms)
    <- against the maximum of ₹9,00,000 (PS router, from a margin of ₹1,00,000)
and forward into the feasibility side:
  <- catchment households (Census PCA) scaled by the growth factor 1.29 (intercensal)
  <- village centroid (SHRUG) with a 10 km haversine radius
  <- dairy demand per household (HCES: MPCE ₹2,946 × 4.9 × 8.44%)
  <- competitors (EC13 block density × households, floored by OSM POIs)

```

Every dataset exists because some link in that chain needs it, and that is the answer to "why exactly these datasets".

4.4 Join keys and granularity decisions

- **shrid (the SHRUG v2.2 ID)** is the single join key across the Census, Antyodaya and spatial files. Use v2.2, not v1.5, because the IDs do not match.
- **Block-level competitor density, village-level application.** We predict a *density* (enterprises per 1,000 households) and then multiply by the village's own household count. Density transfers across spatial scales; raw counts do not. This is the answer to the spatial-downscaling objection.
- **EC13 per-category apportionment** is the honest approximation: EC13 publishes total establishments per block, not per category. We apportion $\text{count_all} \times (\text{emp_shric_category} / \text{emp_all})$ and ship a *method_note* that says so. The SHRUG mapping (dairy is code 7, flour mill is code 8) is documented judgement; **poultry is deliberately unmapped** (SHRUG has no livestock code), and the code falls back to the observed OSM count with a stated note rather than guessing.
- **The HCES CV** (coefficient of variation, 0.461 for Jharkhand dairy) is computed from the published fractile distribution, and that is where our demand *bands* come from.

4.5 Validation (what makes the claim stick)

- The populations of all 29,480 JH villages sum to 25,054,425 against the published rural Census total of 25,055,073, an error of **0.003%** (asserted in `test_real_dataset.py`).
- Every village record passes pydantic validation with JH-bounded geography.
- The committed manifest declares per-state village and POI counts with `synthetic: false`; per-source truth is read from each file's `_meta`, never hardcoded.
- **[Planned]** credibility item: ground-truth shop counts in a Ranchi block compared against our estimates, a claim no competing team can make.

4.6 Known data gaps (say them before the judge does)

Gap	Prepared position
Census 2011 is fifteen years old	It is the only village-level dataset that exists in India. We scale forward with district growth rates and anchor demand to HCES 2023-24. Census 2027 is underway.
OSM rural sparsity (120 POIs in all of rural JH)	An observed count is a floor on competitors, never a census. The EC13 estimator exists because of this, and its output is a range, not a point.
No seasonality history from Agmarknet	The chart is omitted on live data rather than estimated.
<code>name_hi</code> is null for real villages	Devanagari names exist in the sample fixture; populating real Hindi names is open.
One state (Jharkhand)	Depth-first pilot: everything is sharded per state (<code>villages/<ST>.json.gz</code>), so adding a state is an ETL run, not a code change.
No business-failure labels anywhere	We stress-test instead of classifying. See section 8.

5. Mathematical models: every formula, with worked numbers

All money math uses `Decimal` (never float), quantized to paise with `ROUND_HALF_UP` (`finance/money.py`). `format_inr` does hand-rolled Indian digit grouping (₹9,00,000), and the frontend mirrors it (`format.ts`) so that "the screen and the report never disagree".

5.1 Scheme router (`finance/router.py`)

Constants: `MARGIN_SHARE = 0.10`, `LOAN_SHARE = 0.90`, Logic A project ceiling **₹1.40 L** with a loan cap of **₹1.25 L**, Logic B project ceiling **₹50.00 L** with a loan cap of **₹45.00 L**, and an out-of-scope margin of **₹5,00,000**.

```
Project cost P = M / 0.10          (the PS formula: it yields the MAXIMUM, not the right, loan)
Maximum loan L = min(0.9 × P, scheme cap)
```

	Logic A (Micro Finance)	Logic B (Term Loan)
Trigger	P up to ₹1.40 L	Above ₹1.40 L, up to ₹50.00 L
Rate (annual)	6.5% , which is 1.625% per quarter	8.0% , which is 2.0% per quarter
Tenure	3 years, i.e. 12 quarters	7 years, i.e. 28 quarters
Moratorium	3 months, i.e. 1 quarter, inside the 3-year window (11 repayment quarters)	6 months, i.e. 2 quarters; 12 months (4 quarters) for plantation and construction activities
SCA on-lending	NSFDC to SCA at 2.5%	at 4.0%
When the cap bites	P of ₹1.4 L would give $0.9P = ₹1.26$ L, so the ₹1.25 L cap binds (<code>capped=true</code>)	the ₹45 L cap binds only for the largest projects (a ₹50 L project may borrow only ₹45 L)

Boundary tests pin all six edges (`test_router.py`): a margin of 13,999 or 14,000 routes to Logic A (P of ₹1,40,000 is exactly the ceiling), 14,001 routes to Logic B, 5,00,000 stays in scope, and 5,00,001 falls **OUT_OF_SCOPE** with referrals to PMEGP, PMMY and Stand-Up India (maximum loan of zero).

Demo persona: a margin of ₹1,00,000 gives $P = ₹10,00,000$, which routes to Logic B with $L = \min(₹9,00,000, ₹45,00,000) = ₹9,00,000$.

5.2 Quarterly amortization: the Indian quarterly rest (`finance/amortization.py`)

```
r = annual rate / 4          (quarterly rest: interest accrues quarterly on the balance)
EMI = L × r × (1+r)^n / ((1+r)^n - 1)  (becomes L/n when r = 0)
n = total quarters - moratorium quarters
```

We chose $r = \text{annual}/4$, the simple quarterly rest convention that Indian cooperative and scheme lending actually uses, rather than effective-annual compounding. This is not trivia; it changes the instalment:

- **Golden vector B:** ₹9,00,000 at 8% ($r = 2\%$ per quarter), a 2-quarter serviced moratorium, 26 instalments of **₹44,729.31**. Repayment-phase interest comes to about ₹2,62,962, plus moratorium interest of $2 \times ₹18,000$.
- **Golden vector A:** ₹1,25,000 (capped) at 6.5% ($r = 1.625\%$ per quarter), a 1-quarter moratorium, 11 instalments of **₹12,501.34**.
- A caution from our own history: an early draft of `PLAN.md` quoted ₹12,471 for vector A, which is what effective-annual compounding gives. **Never quote ₹12,471**. The correct, tested number is ₹12,501.34, consistent with vector B's convention.

Schedule invariants are tested: every row satisfies `closing = opening + interest - instalment`; the principal column adds up to the amortized principal; the final closing balance is zero; the **last instalment absorbs the rounding residue**, so the schedule closes to the paisa; and repayment-phase interest falls monotonically.

5.3 Moratorium: both treatments, always

`amortize()` implements both treatments and the report shows both side by side:

- **SERVICED**: pay the interest during the moratorium and the principal stays untouched. The instalment equals the interest and the balance is flat.
- **CAPITALIZED**: pay nothing and the interest accrues into the principal. The closing balance is the opening balance plus interest, so the amortized principal becomes $L(1+r)^m$; for example $₹9,00,000 \times 1.0404 = ₹9,36,360$ after two quarters at 2%, with higher instalments afterwards.

The frontend deliberately does **not** ask the user which mode applies: exposing a moratorium choice "would add a piece of financial jargon to the form for a decision almost no first-time borrower is equipped to make" (`urlState.ts`). The report shows the recommended schedule and one extra sentence: *"What if you pay the interest later instead? ₹X a quarter (capitalized)."*

5.4 DSCR right-sizing, the product thesis (`finance/rightsizing.py`)

```
Annual NOI (year) = (monthly revenue × factor - monthly opex) × 12
DSCR (year)       = NOI (year) / debt service (year)
UNBOUNDED_DSCR   = 999          (sentinel for a year with zero debt service)
```

The **largest serviceable loan** is found by bisection (the tested premise: DSCR falls monotonically as the loan grows; tolerance ₹100). Four candidate ceilings are computed and the **minimum wins**:

Constraint	Definition	Threshold
<code>SCHEME_CAP</code>	the router's maximum loan	hard limit
<code>CAPITAL_NEED</code>	what the unit actually costs, minus the margin	hard limit
<code>DSCR</code>	base-case coverage	at least 1.5 (the NABARD appraisal norm)
<code>STRESS</code>	coverage at revenue -15%	at least 1.0 (the floor)

The tie-break rule is a code comment worth quoting: hard limits outrank soft ones, in the order `SCHEME_CAP`, then `CAPITAL_NEED`, then `STRESS`, then `DSCR`, because "a cash-flow constraint that merely saturates at the ceiling has not bound anything."

Rounding: the recommendation is floored to the nearest ₹1,000, **except** when `CAPITAL_NEED` binds, where flooring would itself manufacture a capital shortfall.

The demo, worked through:

- The base NOI is $(₹29,400 - ₹21,600) \times 12 = \textbf{₹93,600 a year}$.
- The maximum loan of ₹9,00,000 carries annual debt service of $4 \times ₹44,729.31 = ₹1,78,917$, so the worst-year DSCR is **0.52**, which fails the 1.5 norm.
- Bisection settles on ₹2,05,000: the instalment is about ₹10,187 per quarter, or ₹40,748 a year, giving a base DSCR of **2.30**. Under the -15% revenue stress the NOI falls to $(29,400 \times 0.85 - 21,600) \times 12 = ₹40,680$, so the stressed DSCR is **1.00**, exactly meeting the floor. The binding constraint is `STRESS`, shown to the user as "limited by what survives a bad year".

- The required capital is ₹4,10,800 (fixed capital of ₹3,46,000 plus three months of working capital at ₹64,800), so the debt need is ₹3,10,800 and the headroom is 9,00,000 - 2,05,000 = **₹6,95,000**, the red-versus-green gap on screen.
- The overborrowing warning is generated verbatim: *"At the maximum permissible loan of ₹9,00,000 the worst-year DSCR is 0.52, below the 1.5 appraisal norm. This is the borrowing level that causes the defaults the scheme is trying to prevent."*

5.5 Stress testing: what replaces an ML failure classifier

The stress runs recompute the same deterministic model at revenue -15% and -30% for **both** the maximum and the recommended loan, reporting per-year DSCR rows. The shape of the result (same arithmetic as the code):

Base case:	profit ₹18,000/mo	EMI ₹4,950	comfortable
Revenue -15%:	profit ₹12,400/mo	EMI ₹4,950	tight
Revenue -30%:	profit ₹7,200/mo	EMI ₹4,950	distress
Maximum loan ₹9L:	profit ₹18,000/mo	EMI ₹14,800	FAILS THE BASE CASE

Why there is no success-prediction model (memorise this answer): no labelled failure data exists for this population, and fabricating labels produces a confident model that is wrong. We stress-test instead. This is a *deliberate* engineering decision, listed in the design document under "deliberately not machine learning" alongside: loan arithmetic (it must be auditable by a bank officer), eligibility verdicts (a language model deciding eligibility is a liability), scheme stacking (graph logic), and the stress tests themselves (recomputation of the same deterministic model).

5.6 The alternatives engine (finance/alternatives.py): never an empty result

`viable_configurations()` loops over each template's candidate sizes (dairy runs from 1 to 6 animals; poultry from 250 to 2,000 birds in steps of 250), re-costs every line item by its scaling rule, and right-sizes each configuration. Results:

- Ranked safest first, by worst-year DSCR descending and then project cost ascending.
- `funded` means the shortfall is zero or negative; `self_financed` means funded with no loan at all; `viable` means funded **and** the NOI is positive and the binding constraint is not NOT_VIABLE (so a money-losing unit is never offered just because the applicant could pay for it).
- **Larger-contribution logic**: when nothing is fundable, we return the closest configuration and set `additional_margin_needed = shortfall`, because "every extra rupee of margin reduces the debt need by a rupee while the serviceable loan is unchanged." For the demo persona, no dairy size is fundable on ₹1L of savings; the closest is the 4-animal unit, which needs **₹48,800 more margin**.
- The counter-intuitive advisory, straight from the code: "a larger unit often needs less of your own money, not more, because its running costs are spread over more output."
- **The phased expansion plan** (built when the template pays family labour): retain the annual surplus after instalments for up to five years until the next-larger unit's cost is covered, and withhold the plan when it is unreachable. Poultry is excluded because it has no labour line, since "treating its surplus as retained earnings would quietly assume the family lives on nothing."

Scaling rules per line item (`cost_templates.py`): `linear` prices per unit (the cows), `fixed` is one purchase (the chaff cutter), and `step` is one purchase covering `step_capacity` units (milk cans per four animals, family labour per four animals). At the base size the identity holds exactly, which is what makes the published per-template figures regression tests:

Template	Fixed capital	Revenue/mo	Opex/mo	Working capital	Required	NOI/yr
dairy_2_animal	₹3,46,000	₹29,400	₹21,600	₹64,800	₹4,10,800	₹93,600
kirana_store	₹1,70,000	₹46,200	₹18,100	₹18,100	₹1,88,100	₹3,37,200

Template	Fixed capital	Revenue/mo	Opex/mo	Working capital	Required	NOI/yr
tailoring_unit	₹95,000	₹43,800	₹27,000	₹54,000	₹1,49,000	₹2,01,600
atta_chakki	₹1,80,000	₹32,200	₹17,100	₹34,200	₹2,14,200	₹1,81,200
backyard_poultry	₹1,29,000	₹39,380	₹34,800	₹69,600	₹1,98,600	₹54,960

5.7 Demand estimation, the fuel for Opportunity Analysis (feasibility/demand.py)

The HCES mean-band estimator bridges the gap between a state average and this village:

```
per household = rural MPCE × household size × category share
low, high     = per household × (1 - CV), per household × (1 + CV)    (plus or minus the coefficient of variation)
market size   = per household × catchment households × addressable share
```

The Jharkhand constants (committed, from HCES 2023-24) are: rural MPCE of **₹2,946** against an all-India rural figure of ₹4,122, household size **4.9**, a dairy share of **8.44%**, a CV of **0.461** and an addressable share of **0.45**, each share shipped with its written rationale. Two declared caveats live in the file's own `method_note`: the category *shares* are all-India rural figures applied to each state's MPCE (state-level shares need the HCES microdata at microdata.gov.in, catalogue 237), and `addressable_share` **is an assumption shipped with its rationale** rather than disguised as data.

The worked per-household dairy spend is $2,946 \times 4.9 \times 0.0844$, which is about **₹1,218 per household per month**, with a band of **₹657 to ₹1,780**. Multiply by the actual catchment households from the Census (scaled by growth), then by 0.45, and you have the addressable market, always reported as a band: *"Estimated ₹X to ₹Y lakh per month of local milk demand."*

Two honesty notes: (a) the point estimate is exactly the "state average applied uniformly" baseline, and the band is what we add on top of it; (b) the formal name for this problem is **small-area estimation** (the Fay-Herriot model), which is our planned upgrade path. Naming it correctly signals that we know the statistics literature.

Opportunity and gap. The facts builder computes `capacity = competitor point × monthly revenue per competitor`, `demand_supply_ratio = TAM / capacity` and `households_per_competitor = households now / competitor point` (`facts/builder.py`). SWOT rules turn these into "underserved market" (ratio 1.15 or higher) and "saturated market" (below 0.9).

5.8 Competitor estimation (feasibility/competitors.py): Competitor Mapping

```
density = EC13 block density (enterprises per 1,000 households, per category)
point   = density × households now / 1000
z       = (density - district mean) / district std          (context and confidence)
point   = max(point, observed OSM POI count)               ("raised_to_observed": an observed count is a hard floor)
band    = [point × 0.65, point × 1.35]                     (BAND_SPREAD = 0.35, floored at the observed count)
```

The method string ships with the estimate: *"Economic Census 2013 block density × households in radius, floored by observed OSM points of interest."* If no EC13 row exists for the block and category (poultry, for instance), the estimator falls back to the observed OSM count with **Confidence.LOW** and an explicit `no_ec13_row` note. The output is a **range, never a point**: "8 to 14 similar businesses within 10 km".

Why the fixed ±35% band: sparse rural data "does not support anything tighter". Why have an estimator at all: OSM undercounting is *systematic*, and it is worse in poorer, remoter and less-literate areas, exactly where our users live; systematic error is learnable. **[Planned]** Model 1 trains LightGBM on Udyam registered-enterprise labels with the OSM POI count as the primary feature and Census demographics as the other features, validated by **holding out entire states** (random splits leak spatial correlation) and reported as MAE against the raw-OSM baseline.

5.9 Threats (feasibility/threats.py)

- **Seasonality:** the CV of monthly mandi arrivals is `standard deviation / mean`, along with a peak-to-trough ratio; arrivals are "strongly seasonal" when the **CV reaches 0.45**. Threats carry severity tags, and swings of four times or more between trough and peak are rated "high".
- **Monsoon dependency:** dairy, poultry and flour mill are the flagged `monsoon_dependent_categories`.
- **Single-buyer dependence:** fewer than two mandis within the radius is high severity, matching the PS's "single-buyer dependence" wording exactly.
- **Road access:** a pucca-road share below 0.6 is medium severity.
- **Price band:** the 12-month modal price at low, mean and high (₹ per quintal), which becomes the price-spread range bar on the report.

5.10 SWOT as a YAML rules engine, not a prompt (feasibility/swot.py)

The rules live in `data/real/swot_rules.yaml` (13 rules, bilingual, each with citations) and are evaluated by a tiny comparator interpreter, **not eval**, so that "a YAML file can never become code execution". The interpreter supports nested `all:` and `any:` blocks. Every `text_en` and `text_hi` line is rendered with `str.format(**metrics)`, so **every number inside a SWOT line is grounded by construction**, and a missing metric never fires a rule. One rule, verbatim:

```
- id: overborrowing_risk
  quadrant: threat
  when: {metric: max_loan_min_dscr, op: "<", value: 1.5}
  text_en: >-
    The scheme formula permits {max_loan_fmt}, at which the worst-year coverage falls to
    {max_loan_min_dscr}. Borrowing the maximum is the single largest risk to this unit.
  cites: [finance_engine, nsfdc_term_loan]
```

The key thresholds: an underserved market when the demand-supply ratio is 1.15 or higher, and a saturated one below 0.9; a large catchment at 8,000 households or more and a thin one below 3,000; banking access when at least three villages in the radius have a bank and no banking access when none does; healthy repayment capacity at 1.5 or higher and overborrowing risk below 1.5; seasonal cashflow when the arrivals CV is 0.45 or higher; skill and road together when literacy is 0.65 or higher and the pucca-road share is 0.6 or higher; a distance to town of 25 km or more; and the scheme concession, which quotes "6.5 to 8% per annum against a market rate of 12 to 18%".

5.11 Village name matching (matching/village_matcher.py): the unglamorous demo-saver

A user says "Ormanjhi"; the ASR writes "Ormanji", "Aurmanjhi", or "ओरमांझी". The matcher:

1. **Transliterates** Devanagari to Latin. It normalises to NFC, handles the seven nukta pairs explicitly ("बेड़ो is Bero, not Bedo") and then maps roughly sixty glyphs.
2. **Hashes phonetically with an Indic Soundex:** it folds aspirate digraphs first (`chh` to `c`, `kh` to `k`, `jh` to `j` and so on), then applies the classic Soundex groups, with `h` and `y` not breaking a run of the same group, so Ormanjhi, Ormanji, Kankay and Raatu hash identically to their variants.
3. **Scores:** `score = 0.55 × fuzzy(rapidfuzz WRatio over name and name_hi) + 0.30 × phonetic match + 0.15 × GPS prior (max(0, 1 - haversine/50km))`.
4. **Returns the top three candidates with human-readable reasons** ("sounds the same", "spelling and pronunciation both match") and **never silently picks one**, because "a wrong village silently poisons every number downstream."

5.12 Small constants worth knowing

- Intercensal growth: Ranchi district **1.29**, default **1.25** (`config.py`).

- Haversine: Earth radius 6,371.0088 km, which the code notes is "good to ~0.5% at village scale".
- `COMFORT_MARGIN = 0.25` : a configuration that clears the 1.5 norm by less than 0.25 is reported as "tight", because 1.51 against a norm of 1.50 is one bad month from trouble.
- Stress factors of 0.85 and 0.70 with a stress floor of 1.0, enforced by default.

6. Features, flows, and the fast-response granularities

6.1 The user journey, from the Ask screen to the report

The ask screen is a **five-step wizard** rather than one form. The design comment explains why: nine controls on one page is "a wall" for a reader who has never filled in a web form, and splitting them across steps costs four extra taps and removes the wall. A UX4G stepper tracks progress; on mobile only the "Step 2 of 5" text shows.

1. **Village.** Type or **speak** (a microphone button, `hi-IN` or `en-IN`); the search debounces at 220 ms and aborts superseded requests; the top three candidates come back with reasons; **explicit confirmation is required**.
2. **Distance.** A radius slider from 3 to 30 km, defaulting to 10.
3. **Business.** Fetched live from the cost-templates endpoint; each radio option shows what the unit costs ("This unit costs about ₹4,10,800"), which is "the single most useful number at this moment".
4. **Savings.** A numeric keypad input; the helper line echoes the amount in grouped Indian digits *as you type*, so "a mistyped extra zero is visible before it becomes a wrong loan".
5. **About you.** Social category (SC, safai karamchari, OBC, EBC or General), optional family income, and the woman-entrepreneur and prior-experience checkboxes.

The report page (its order is the argument). It opens with the provenance banner, then the answer itself: the LoanComparison card shows the recommended loan first, in green, and the scheme maximum second, in red, because "presenting the max first is exactly the framing that causes over-borrowing". Warnings follow and are never collapsed, because "these are the reasons someone defaults". Next comes the scheme and eligibility card with the documents checklist and the SCA address. Then a "Why this amount?" accordion walks through the headline, the loan structure, the stress panel with the DSCR chart, the repayment schedule alongside the alternate moratorium treatment, market reach as village chips with distances, competition as two range bars, the SWOT grid, and threats with the seasonality chart when history data exists. A provenance drawer button ("Where did that number come from?") and a data note close the page, with a footer stating how many figures were checked against the facts object and which narrator lane produced the prose.

Out of scope (a margin above ₹5L): no report. The screen says *"This scheme cannot fund this business"* and shows the comparison schemes instead. Running the normal report here produced "You should borrow ₹0" under a green tick with a worst-year DSCR of 999.00, which reads as an approval for nothing.

Print and PDF. A dedicated print stylesheet re-points dark tokens to light, hides the screen chrome, and **forces all collapsed accordions open**, on the reasoning that "progressive disclosure is a screen affordance; this document is what they hand to the bank."

6.2 The six PS parameters, from data to code to UI

PS parameter	Data ground	Code	UI
Market Reach	Census PCA households, SHRUG centroids, Antyodaya amenities and the growth factor	<code>feasibility/market_reach.py</code> : <code>villages_within(10 km)</code> , the household sum, bank, mandi and road shares, income segments	the market-reach panel and village chips
Opportunity Analysis	HCES demand bands against competitor capacity	<code>demand.py</code> , then the demand-supply ratio and households per competitor	the headline and the SWOT opportunity rules
SWOT	All of the above	<code>swot.py</code> with 13 YAML rules	<code>SwotGrid</code> (a 2 by 2 with tone, icon and text, so colour never carries meaning alone)

PS parameter	Data ground	Code	UI
Threats Identification	Antyodaya mandis and roads, Agmarknet arrivals and prices	<code>threats.py</code> (seasonality CV, single buyer, monsoon, roads)	severity-tagged threats and the seasonality chart when history exists
Competitor Mapping	EC13 block densities and OSM POIs	<code>competitors.py</code> (density times households, z-score, OSM floor, $\pm 35\%$ band)	the competition range bar ("Fairly sure" / "Roughly right" / "Rough guess")
Product Market Value	Agmarknet modal prices and HCES income segments (the bottom 30% at ₹1,717 MPCE up to the top 10% at ₹6,099)	the price band in <code>threats.py</code> plus purchasing-power segmentation	the price-spread range bar and scheme-concession framing

6.3 Voice and ASR: what is real and what is designed

- **[Built]** The browser **Web Speech API** (`MicButton.tsx`): the language is set to `hi-IN` or `en-IN`, there are no interim results, and the transcript feeds the village search. Unsupported browsers get an inline message, never a dead button ("Voice input is not available in this browser. Please type instead." and its Hindi equivalent). The button always shows a text label ("a microphone glyph alone is not a word"), carries an `aria-pressed` toggle and a "Listening..." state.
- **[Built]** The *real* voice-enabling work is what makes noisy ASR output usable: the phonetic village matcher (section 5.11) absorbs transcription noise, and all five tested ASR-noise variants of "Ormanjhi" rank first.
- **[Planned]** The offline voice stack: **Silero VAD** (about 1 MB; it detects the end of an utterance so voice input feels instant), then **IndicConformer or IndicWhisper in INT8** for speech to text in Indian languages (ONNX Runtime), then an **intent and slot classifier** over IndicBERT or LaBSE embeddings (intent classes: `feasibility_check`, `scheme_eligibility`, `application_process`, `rejection_reason` and `loan_calculation`; slots: business category, margin capital, village, caste category and state), and finally **Indic-TTS or Piper** to read the report aloud. The target is fully offline operation at a bank branch or a Common Service Centre.

6.4 The narration ladder and the anti-hallucination stack (narration/)

Three lanes are tried in order, and **the bottom rung always works**:

1. **Cloud LLM**. It emits schema-constrained JSON with the section ids pinned, at low effort.
2. **Local Llama 3.1 8B Instruct Q4_K_M** (llama.cpp, temperature 0.2, "paraphrase, do not invent") with a **GBNF grammar generated from the report's own section ids**: the sampler is *physically unable* to emit a section that is missing, reordered or invented. The weights (about 4.7 GB) are not shipped, and the lane advertises itself as unavailable until configured.
3. **Jinja template narrator**. Deterministic prose with `StrictUndefined` (a missing facts key raises rather than silently blanking), with the English and Hindi texts kept adjacent, because "a translation that drifts from its source is a correctness bug".

Every lane's output is post-validated by the **numeric-grounding validator** (`narration/validator.py`):

- It extracts numbers in Indian notation (₹9,00,000, 44,729.31, 8.0%, ₹9.00 L, 1.25 Cr, "2 lakh") and computes each number's **resolution**, the place value of the last written digit: "₹44,729" resolves to 1 and "₹14.33 L" to 1,000.
- The allowed set is the facts object's `allowed_numbers()` (the entire numeric index, both amortization schedules, cost lines, DSCR rows, seasonality values, neighbouring villages and income segments, all quantized to 0.01), plus the structural constants 10, 15, 30, 90, 100 and 1,000 (the margin split, the stress percentages and the per-thousand denominator), plus source years, small ordinals, and the numbers inside approved verbatim strings.
- A number passes when its distance from some allowed value is at most `max(0.01, resolution × 0.5)`: ₹44,729 tolerates fifty paise, while "₹14.33 L" tolerates ₹500.

- The catch rate (rejections over generations) is exposed at `/api/v1/metrics`. "Our numeric validator caught and corrected X% of generations" is a concrete, honest anti-hallucination result to quote.

The three defence layers: first, **structure**, where the grammar or schema makes malformed output impossible; second, **numbers**, where every figure must appear in the facts object; third, the **paraphrase contract**, where the LLM receives the deterministic draft and may only rephrase it. Headings, citations and chart data always come from the deterministic draft, and on any failure the template draft ships, so **there is no input under which a generative lane makes the output worse than the offline one**.

One known limit, stated in the code itself: the validator catches *invented* numbers, not *misattributed* ones. That is why eligibility verdicts come only from the rule engine, with verdict fidelity targeted at exactly 100%.

6.5 The bilingual strategy

- Report prose is **narrated server-side in the requested language**; switching the language re-runs the advisory request. Client-side translation of report text is treated as a correctness bug.
- UI chrome uses two hand-maintained string tables (`format.ts`, `T.en` and `T.hi`). Two languages only, "so a pair of buttons beats a dropdown".
- The `<html lang>` attribute is synced on every switch (otherwise screen readers announced Devanagari with an English voice), and the language lives in the URL, so a shared link opens in the language it was shared in.
- Noto Sans Devanagari is loaded with a `unicode-range`, so an English session never downloads the roughly 50 KB of Hindi font files.

6.6 Why it is fast: the performance granularities

Measured on a normal machine with the real Jharkhand data: a cold loader init takes about 159 ms; the first shard load plus village lookup about 216 ms; the 10 km neighbour scan over **29,480 villages** about 62 to 80 ms; and the **entire facts build about 490 ms**. The levers:

1. **Lazy, gzipped, per-state shards**. A cold start reads only the manifest and the metadata, roughly 150 KB, instead of about 150 MB of India-wide JSON. The requested state's shard is parsed once, cached, and indexed by `shrid` as it loads.
2. **`lru_cache` on every expensive singleton**: settings, the data source, cost templates, SWOT rules, the Jinja environment and the local GGUF model.
3. **Precomputed aggregates committed to disk**. EC13 block densities and district means and standard deviations, HCES state profiles with CVs, income segments and category-to-commodity proxies are all computed by the ETL, so the request path is dictionary lookups and arithmetic.
4. **Zero network at request time**. Every dataset is committed; the data.gov.in key is used by the ETL only. The demo works with the network disabled.
5. **Deliberately O(N) small-N geometry**. A plain haversine scan; PostGIS is the planned replacement "the moment real polygons land", but at 29,480 rows a linear scan beats the complexity of deploying a spatial index.
6. **One synchronous POST**. No polling or streaming; the whole pipeline returns at once. The Vercel function budget is `maxDuration: 30`.
7. **Frozen pydantic models with `Decimal`**. Deterministic, hashable and cache-friendly, and the formatting cooperates with the validator's half-resolution slack.
8. **Frontend discipline**. Money travels as strings (floats only for chart pixels), effects are keyed by serialized input, and typeahead debounces and aborts superseded requests.

6.7 The API surface

Endpoint	Method	Purpose
/api/v1/advisory	POST	The full pipeline: facts, report and validation. 404 on an unknown village or template; 422 on bad input.
/api/v1/finance/structure	POST	Finance only: the scheme route, right-sizing and both schedules (the golden vector B is reproduced over HTTP in the tests).
/api/v1/villages/search	GET	Ranked village matches (q, state, limit defaulting to 3).
/api/v1/cost-templates	GET	The five templates with derived totals (feeds the Ask wizard's category list).
/api/v1/schemes	GET	Corporations and comparison schemes.
/api/v1/metrics	GET	The validator catch rate, the active narrator lane, the estimator, and the data source ("live" or "sample").

7. What we did extra, and why it shines

1. **The facts contract with per-figure provenance** (`facts/builder.py`, `provenance.py`). Every one of the roughly 70 computed figures carries a pointer to its source dataset, drawn from 15 registered sources, each with publisher, URL, year and an honesty note (OSM's reads "a POI count is a floor on competitors, not a census"). Tests assert that the provenance map covers exactly the numeric index. *Why it shines*: "Where did that number come from?" is answered for every number on screen, before it is asked.
2. **Bands, never point estimates**. Competitors carry a $\pm 35\%$ band, demand carries the $\pm CV$ band, DSCR arrives as a series, and confidence is translated into plain language ("Fairly sure" / "Roughly right" / "Rough guess"). *Why*: honest ranges are the difference between an advisor and a fortune teller.
3. **Grammar-constrained decoding, numeric grounding and the paraphrase contract**. Malformed output is structurally impossible; invented numbers are caught with context and counted. *Why*: it is the anti-"ChatGPT wrapper" answer, demonstrable live.
4. **Maximum versus recommended, visually argued**. The green bar (recommended) leads and the red bar (maximum) trails; the worst-year DSCR is footnoted in red when it fails; the binding constraint is translated ("limited by what survives a bad year"). *Why*: the whole thesis in one card.
5. **The alternatives engine**. A smaller-unit table, the larger-contribution amount and the phased expansion, with never an empty result and never a money-losing suggestion. *Why*: most teams stop at "ineligible".
6. **Honesty UX**. A synthetic-data banner, per-source official and sample tags, the seasonality chart correctly absent on live data, the "no_ec13_row" case said in words, and UNBOUNDED_DSCR rendered as "no repayment due this year" (never "999.00"). *Why*: credibility is the moat.
7. **The out-of-scope screen** instead of a ₹0-loan approval page, with referrals to PMEGP, PMMY and Stand-Up India. *Why*: knowing when to say "this scheme can't help you" is advisory maturity.
8. **UX4G (the Government of India design system), enforced**. `ux4g-web-components@2.1.0` plus a build-time verifier that fails the build on any undefined `ux4g-*` class, any non-system class, or any typescale misuse; 44px touch targets (raised above the design system's own 36px, and documented); dark mode; a text scale of 1, 1.15 and 1.3; and the print stylesheet. *Why*: a ministry audience sees a government-grade portal, not a startup mock.
9. **Golden test vectors to the paisa, with 248 tests and the decision layer pinned at 100% coverage**. ₹44,729.31 and ₹12,501.34 are asserted over HTTP; schedule invariants, boundary margins and the demo persona's numbers are asserted as regression tests. *Why*: "auditable, not predicted" is proven, not claimed.
10. **The em-dash lint gate** (`test_content_style.py`, 26 tests): no em dash in any user-visible copy, because "SIH evaluators actively penalise AI slop". *Why*: it signals that we sweat the authenticity of every pixel of text.
11. **The village matcher with confirmable candidates**. Phonetic plus fuzzy plus a GPS prior, human-readable reasons, never auto-selected. *Why*: it is the unglamorous piece every demo breaks on, and ours is tested against ASR noise.
12. **Runs fully offline, deployable in one click**. Committed datasets, no request-time network, a quantized local LLM lane, Docker Compose with a same-origin nginx proxy, and a single Vercel deployment. *Why*: "assume venue Wi-Fi is unusable" is a planning principle, and we took it.

8. Honest gaps and judge questions

Not built (know this cold; volunteer it before being asked)

- No ML model is trained yet. Estimation is deterministic v0; the LightGBM estimator slot exists and is one config change away.
- No server-side ASR or TTS. The browser Web Speech API stands in, and the IndicConformer stack is planned.
- One state (Jharkhand); the Hindi name field is empty for real villages; Udyam is not ingested yet.
- No scheme stacking graph (comparison cards exist), no PostGIS, pgvector or RAG store, and no server-side PDF (browser print).
- Agmarknet publishes no history, so there is no live seasonality.

Prepared answers

Judge asks	We answer
Where did that number come from?	Name the dataset and open the provenance drawer: every figure traces to Census, SHRUG, EC13, HCES, OSM, Agmarknet or an official scheme portal.
Isn't this a ChatGPT wrapper?	The LLM produces no numbers and makes no decisions. It receives a locked facts object and is validated against it. Demo: the validator catching an invented figure, live.
Census 2011 is outdated	It is the only village-level dataset in India. We scale forward with district growth rates and anchor demand to HCES 2023-24.
Why no success-prediction model?	No labelled failure data exists for this population; fabricating labels yields a confident model that is wrong. We stress-test instead.
How accurate is the competitor estimate?	We report a range with a stated method (EC13 density times households, floored by observed OSM), and the planned model reports MAE against the raw-OSM baseline on held-out states.
Does it work without internet?	Yes: committed data, no request-time network, and an optional fully-local LLM lane. Demo with the network off.
What about the PS's own formula?	We implement it exactly in the router, and then answer the question it should have asked: the maximum and the serviceable loan, side by side.
Why is the maximum loan red?	Because at ₹9,00,000 the worst-year coverage is 0.52, which is the borrowing level that causes the defaults this scheme exists to prevent.

9. Appendix

9.1 Key file map

docs/STUDY_GUIDE.md	this guide
docs/PRD-COVERAGE.md	the built, partial and not-built matrix against the plan
README.md	the current-state overview and run commands
PLAN.md	master plan v2 (verified scheme facts, timeline, risks)
SIH26091_Models_Architecture_Workflow.docx	the design document with the ML roadmap
backend/setubiz/	
finance/	router.py, amortization.py, rightsizing.py, alternatives.py, cost_templates.py, money.py
feasibility/	market_reach.py, demand.py, competitors.py, threats.py, swot.py
eligibility/	rules.py
facts/	builder.py, provenance.py (the contract)
narration/	pipeline.py, validator.py, llm_narrator.py, local_narrator.py, template_narrator.py, paraphrase.py
matching/	village_matcher.py
data/	loader.py, geo.py, data/real/, data/sample/
api/	main.py, routes.py, schemas.py
backend/tests/	13 modules, 248 tests: amortization, router, rightsizing, alternatives, feasibility, eligibility, facts_contract, validator, local_narrator, matcher, api, real_dataset, content_style
etl/	build_villages, build_ec13_density, build_hces_demand, fetch_pois, fetch_arrivals, build_manifest
frontend/src/	pages/Ask.tsx, pages/Report.tsx, components/, api.ts, format.ts, urlState.ts, app.css

9.2 Glossary

Term	Meaning
DSCR	Debt Service Coverage Ratio: net operating income divided by debt service for a period. The NABARD appraisal norm is 1.5 or higher. Our core right-sizing metric.
Moratorium	A repayment holiday at the start of the loan. Serviced means interest-only; capitalized means the interest is added to the principal.
Quarterly rest	Interest accrues quarterly on the outstanding balance, so $r = \text{annual rate} / 4$.
Margin money	The promoter contribution, 10% of the project cost (the PS's "Available Margin Capital").
Logic A and Logic B	The NSFDC Micro Finance scheme (project cost up to ₹1.4 L at 6.5% for 3 years with a 3-month moratorium and a ₹1.25 L loan cap) and the Term Loan scheme (₹1.4 L to ₹50 L at 8% for 7 years with a 6-month moratorium and a ₹45 L cap), verbatim from official portals.
SCA	State Channelizing Agency, the corporation's state arm that actually on-lends (for example JSCCDC at Dhurwa, Ranchi).
NSFDC, NSKFDC, NBCFDC	The corporations for SC applicants (a ₹5L income ceiling with effect from 07-01-2026), safai karamcharis (no ceiling) and OBC or EBC applicants (a ₹3L ceiling).
MPCE	Monthly Per-Capita Consumption Expenditure, from the HCES. Rural Jharkhand is ₹2,946.
HCES	The Household Consumption Expenditure Survey 2023-24 (NSO).
EC13 and SHRIC	The Economic Census 2013, and the SHRIC industry codes used to apportion block enterprise counts to categories.
SHRUG and shrid	The Socioeconomic High-resolution Rural-Urban Geographic dataset (Development Data Lab) and its village ID, our join key.
PCA	The Primary Census Abstract, the Census 2011 village tables.
TAM	Total addressable market: here, the monthly category demand across the 10 km catchment.

Term	Meaning
GBNF	The llama.cpp grammar format that constrains generation to a valid structure, so malformed output becomes impossible.
UX4G	The Government of India design system (v3.1), our frontend component and token contract.
Binding constraint	Which ceiling actually limited the recommended loan: scheme cap, capital need, stress, or DSCR.

9.3 A three-minute demo script

1. **The form** (30 seconds). Type "ormanji" and confirm Ormanjhi from the candidates, pointing out that the app never silently picks. Mention that the microphone works in Hindi. Enter savings of ₹1,00,000, pick dairy, keep the defaults.
2. **The answer** (45 seconds). Green ₹2,05,000 against red ₹9,00,000. Read the red footnote aloud: worst-year DSCR 0.52. Binding constraint: "limited by what survives a bad year". Then the line: "Everyone else built a calculator for the most you can borrow. We built one for the most you can repay."
3. **Why this amount** (45 seconds). Open the stress panel and show the minus-15% case landing exactly at 1.0. Show the repayment schedule with both moratorium treatments. Open the provenance drawer: "every one of these figures has a named source".
4. **The honest alternative** (30 seconds). The alternatives section shows that no dairy size fits ₹1 lakh, that the 4-animal unit needs ₹48,800 more, and that a phased plan exists. "The advice is to wait or save, not to default."
5. **Close** (30 seconds). Switch to Hindi (the server re-narrates), print the sheet for the bank visit, and open `/api/v1/metrics` to show `data_source: "live"` and the validator catch rate. All of it works offline.

Study guide generated from a full read of the codebase on 2026-09-11. Line numbers drift; function names and constants are stable. If a number here disagrees with a test, the test wins.